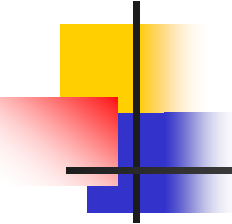


算法设计与分析

Analysis and Design of Algorithm

Lesson 07



要点回顾

■ 动态规划概述

■ 最优化问题

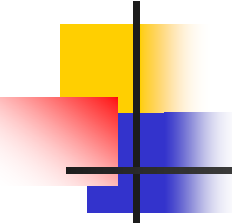
- 概念：约束条件、可行解、目标函数、最优解
- 例子：POS机找零钱

■ 最优性原理/优化原则

- 最优子结构性性质
- 多阶段决策过程

■ 基本步骤

- 分段、分析、求解（自底向上）
- 实例：多源多目标最短路径



动态规划法基本步骤

动态规划法设计算法一般分成三个阶段：

1. **分段**：将原问题分解为若干个相互重叠的子问题；
2. **分析**：分析问题是否满足最优性原理或者优化原则，找出动态规划函数的递推式；
3. **求解**：利用递推式**自底向上**计算，实现动态规划过程。

记住：动态规划法利用问题的最优性原理，以**自底向上**的方式从子问题的最优解**逐步构造**出整个问题的最优解。

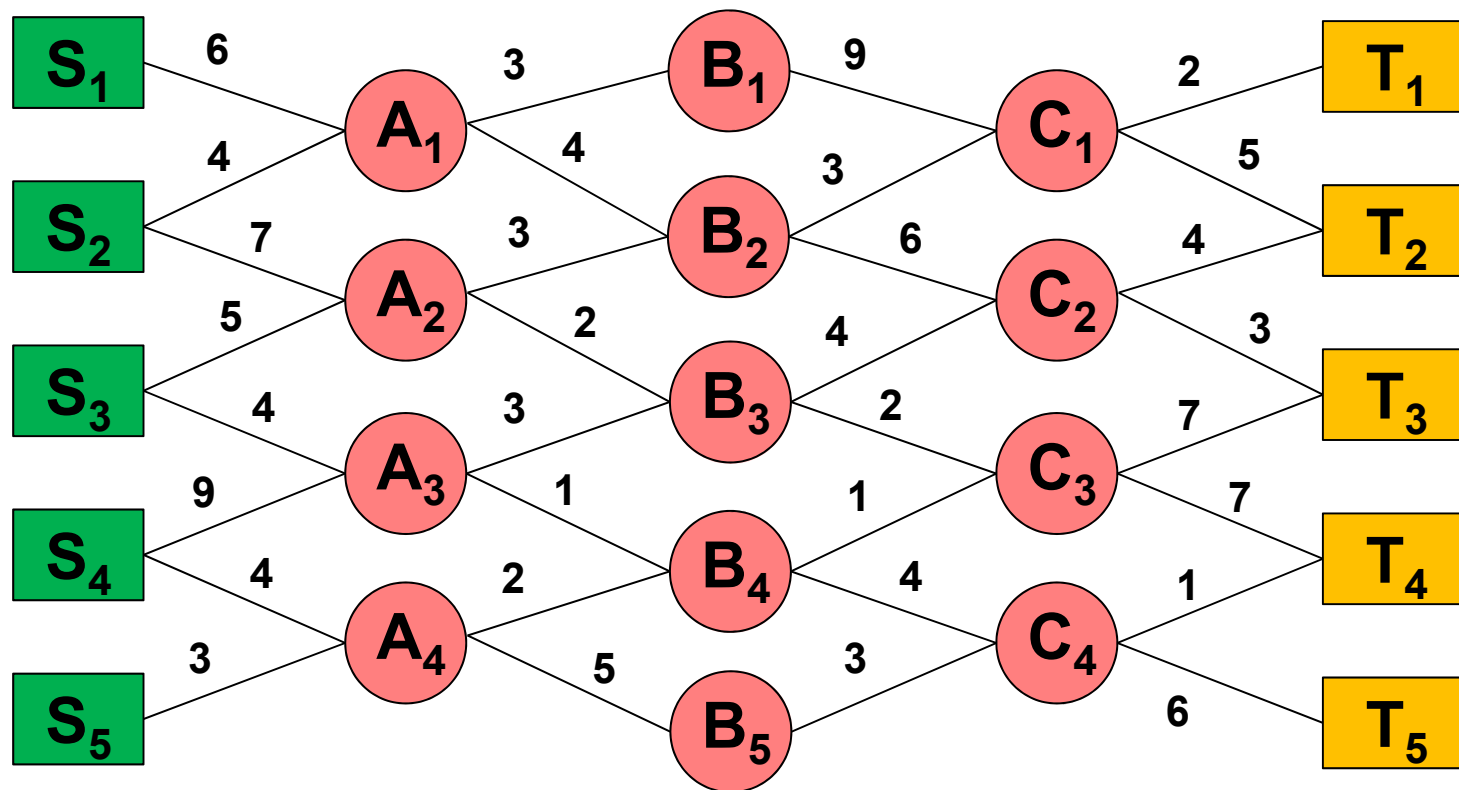
例：最短路径问题

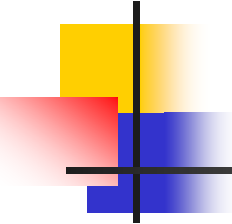
- **问题：** 找任意起点到任意终点的一条最短路径
- **输入：**
 - 起点集合 $\{S_1, S_2, \dots, S_n\}$
 - 终点集合 $\{T_1, T_2, \dots, T_m\}$
 - 中间结点集合，边集，对于任意边 e 有长度
- **输出：** 一条从起点到终点的最短路



例：最短路径问题

■ 一个实例



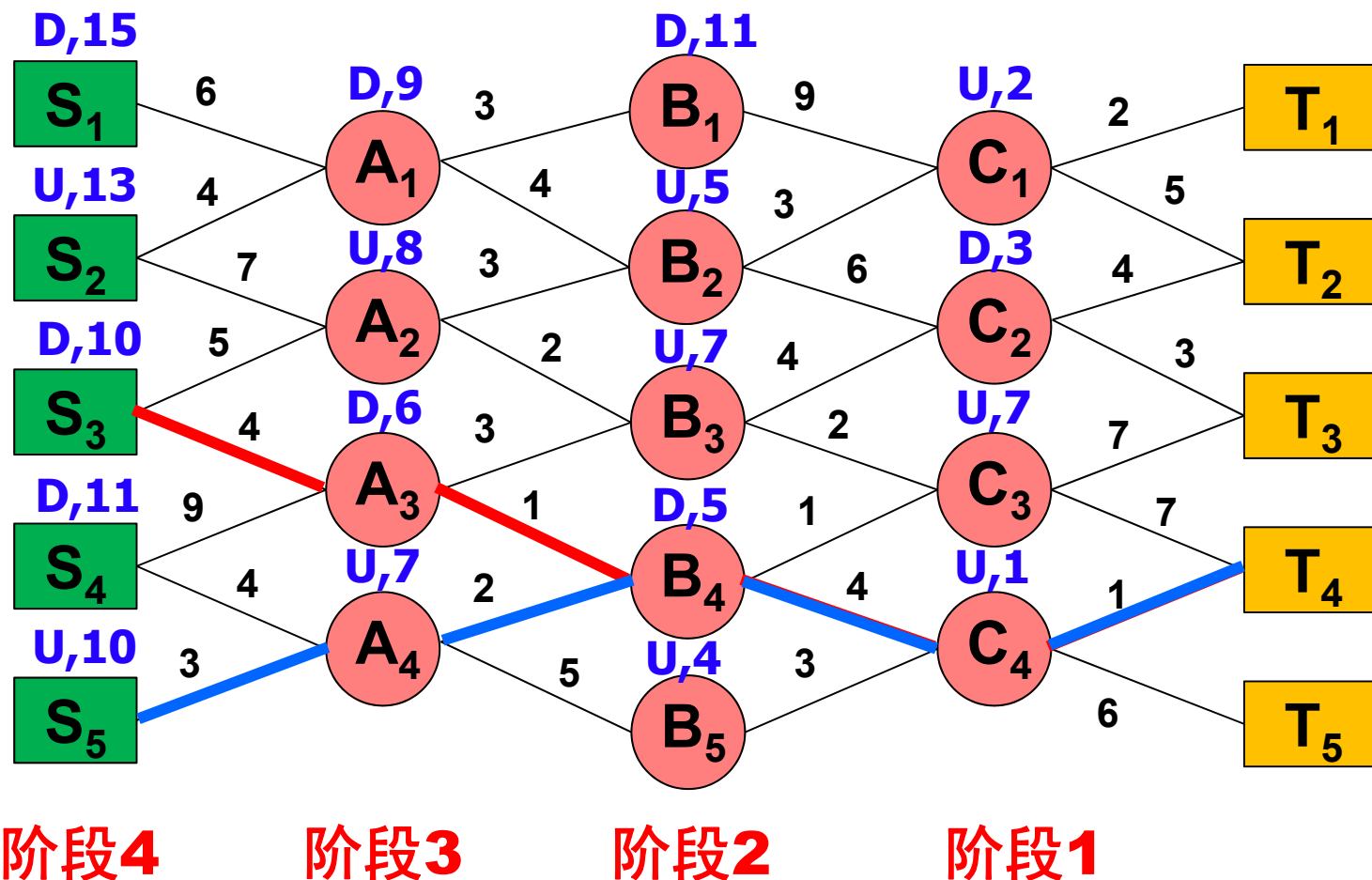


例：最短路径问题

- **蛮力算法/穷举法：** 考察每一条从某个起点到某个终点的路径，计算长度，从中找出最短路径。
- 在上述实例中，如果网络的层数为 n ，那么路径条数将接近 2^n
- **动态规划算法：** 多阶段决策过程。每一步求解的问题是后面阶段求解问题的子问题。每步决策将依赖于以前步骤的决策结果。

例：最短路径问题

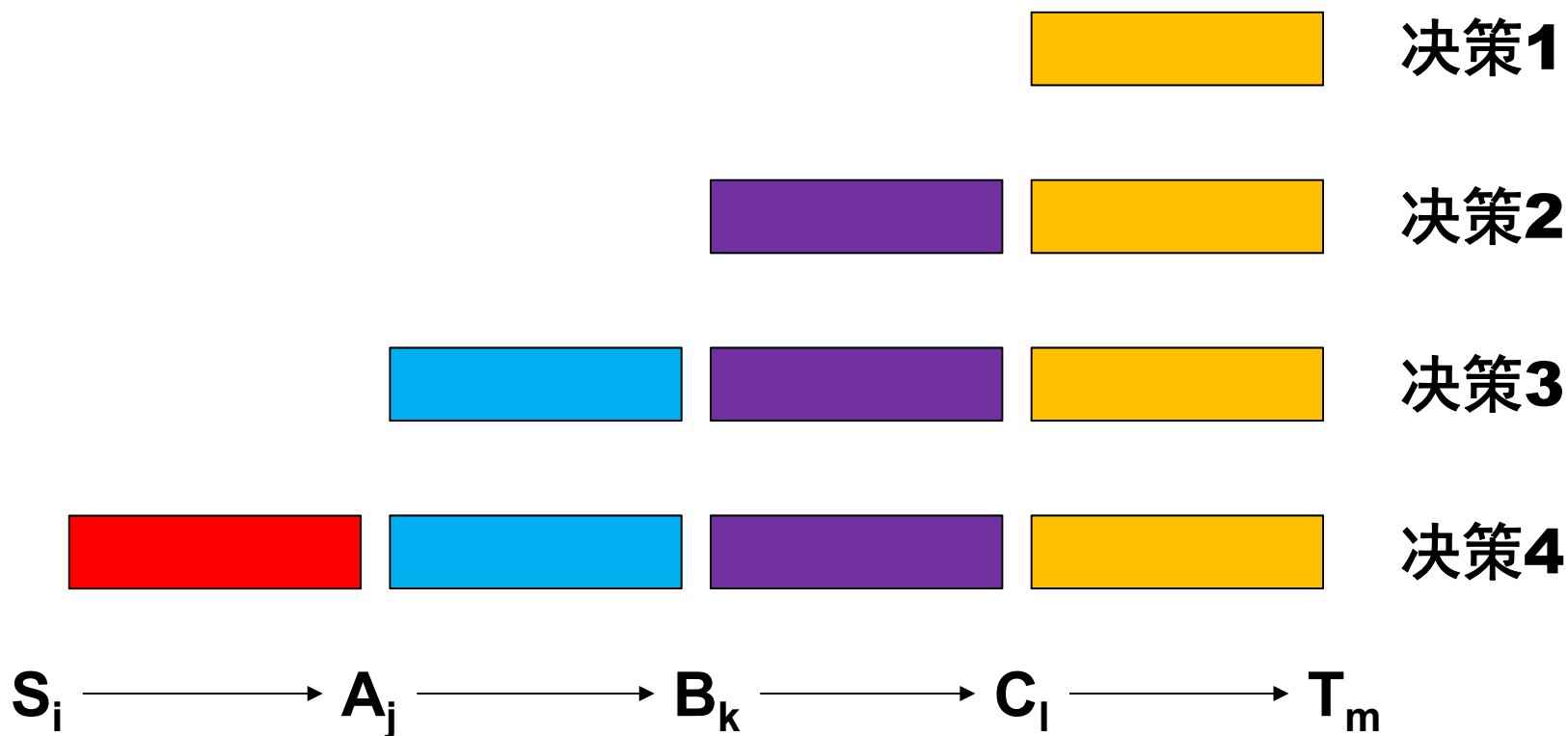
■ 动态规划求解



例：最短路径问题

■ 子问题界定

后边界不变，前边界前移



例：最短路径问题

■ 最短路径的依赖关系

$$F(C_l) = \min_m \{C_l T_m\}$$

决策1

$$F(B_k) = \min_l \{B_k C_l + F(C_l)\}$$

决策2

$$F(A_j) = \min_k \{A_j B_k + F(B_k)\}$$

决策3

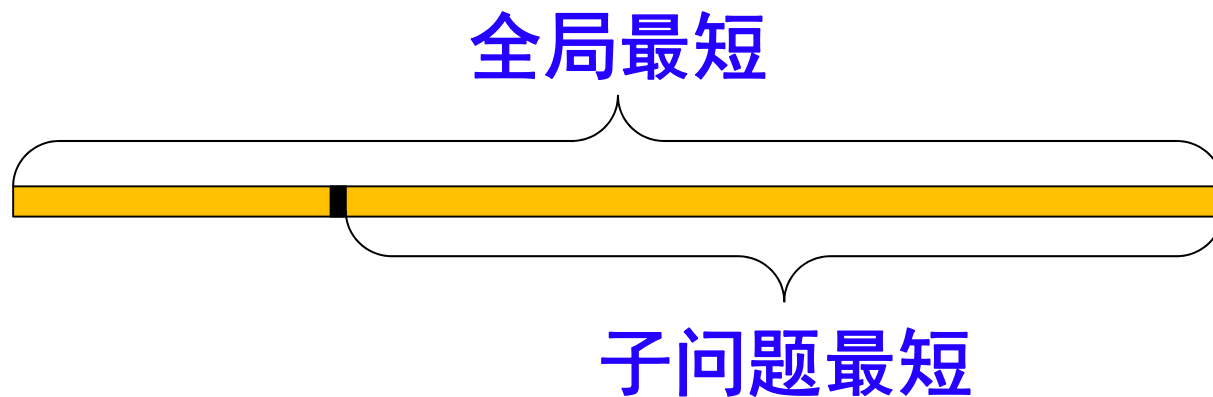
$$F(S_i) = \min_j \{S_i A_j + F(A_j)\}$$

决策4

优化函数值之间存在依赖关系

优化原则：最优子结构性质

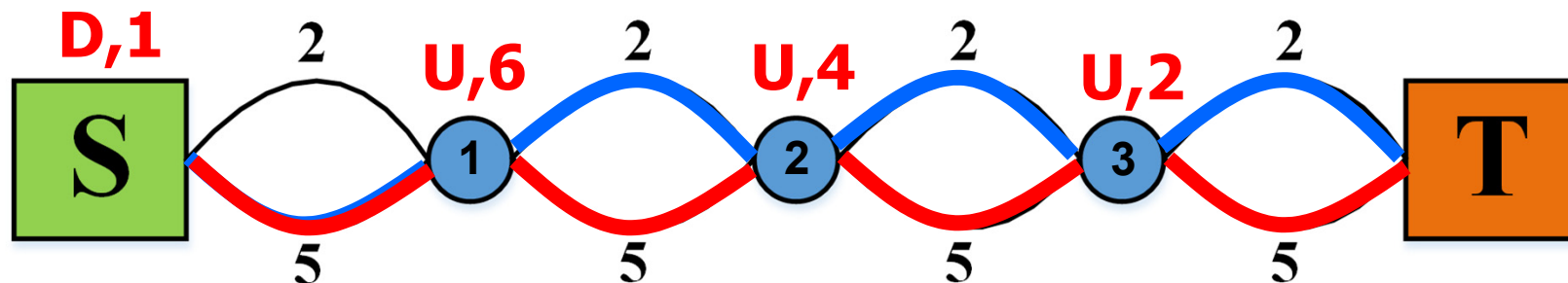
- **优化函数的特点：**任何最短路的子路径相对于子问题始、终点最短



- **优化原则：**一个最优决策序列的任何子序列本身一定是相对于子序列的初始和结束状态的最优决策序列

一个反例

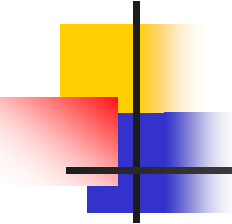
- 求总长模10的最短路径



➔ 动态规划算法的解：下、上、上、上

- 最优解：下、下、下、下

不满足优化原则，不能用动态规划！！！！

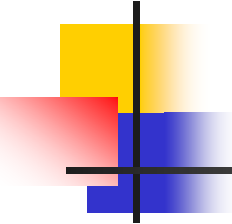


小结

动态规划(Dynamic Programming)

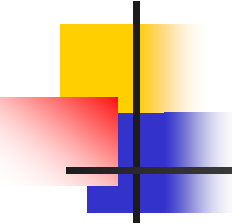
- 求解过程是多阶段决策过程，每步处理一个子问题，可用于求解组合优化问题
- 适用条件：问题要满足优化原则或者最优子结构性质，即：一个最优决策序列的任何子序列本身一定是相对于子序列的初始和结束状态的最优决策序列

动态规划算法设计



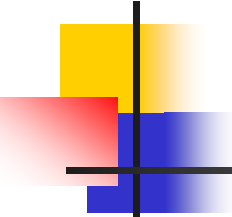
动态规划设计要素

1. 问题建模，优化的目标函数是什么？约束条件是什么？
2. 如何划分子问题？（**边界**）
3. 问题的优化函数值与子问题的优化函数值存在着什么依赖关系？（**递推方程**）
4. 是否满足优化原则/最优子结构性质？
5. 最小子问题怎样界定？其优化函数值，即初值等于什么？



动态规划设计要素

- 归纳起来就是：
 1. **建模**：目标函数、约束条件
 2. **分段**：确定子问题的**边界**
 3. **分析**：原始问题与子问题之间的**依赖关系**
 4. **判断**：最优子结构性质
 5. **求解**：先定最小子问题（初值），**自底向上**求解
- 动态规划算法实例
 1. 矩阵连乘问题
 2. 最长公共子序列
 3. 背包问题



矩阵连乘背景

- 给定 n 个矩阵 $\{A_1, A_2, \dots, A_n\}$ ，其中 A_i 与 A_{i+1} 是可乘的, $i = 1, 2, \dots, n-1$ 。考察这 n 个矩阵的连乘积

$$A_1 A_2 \dots A_n$$

- 由于矩阵乘法**满足结合律**，所以计算矩阵的连乘可以有許多不同的计算次序。这种计算次序可以用**加括号的方式**来确定。
- 若一个矩阵连乘积的计算次序完全确定，也就是说该连乘积已完全加括号，则可以依此次序反复调用2个矩阵相乘的标准算法计算出矩阵连乘积。

回顾：矩阵相乘乘法次数

- 矩阵A： i 行 j 列，B： j 行 k 列，以元素相乘作基本运算，计算AB的工作量

$$\begin{bmatrix} \cdots \\ a_{t1} \cdots a_{tj} \\ \cdots \end{bmatrix} \begin{bmatrix} b_{1s} \\ \vdots \\ b_{js} \end{bmatrix} = \begin{bmatrix} c_{ts} \end{bmatrix}$$

$$c_{ts} = a_{t1} \times b_{1s} + \cdots + a_{tj} \times b_{js}$$

- AB： i 行 k 列的矩阵，计算每个元素需要作 j 次乘法，总计乘法次数为： $ik \times j = ijk$

矩阵连乘背景

- 给定 n 个矩阵 $\{A_1, A_2, \dots, A_n\}$ ，其中 A_i 与 A_{i+1} 是可乘的, $i = 1, 2, \dots, n-1$ 。考察这 n 个矩阵的连乘积

$$A_1 A_2 \dots A_n$$

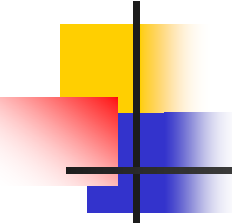
- 由于矩阵乘法**满足结合律**，所以计算矩阵的连乘可以有許多不同的计算次序。这种计算次序可以用**加括号的方式**来确定。

例子：设有四个矩阵 A, B, C, D ，它们的维数分别是：

$$A = 50 \times 10 \quad B = 10 \times 40 \quad C = 40 \times 30 \quad D = 30 \times 5$$

$$\begin{array}{ccc} (((AB)C)D) & (A(B(CD))) & ((AB)(CD)) \\ \mathbf{87500} & \mathbf{10500} & \mathbf{36000} \end{array}$$

$$\begin{array}{cc} ((A(BC))D) & (A((BC)D)) \\ \mathbf{34500} & \mathbf{16000} \end{array}$$



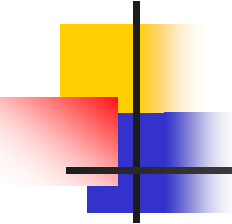
矩阵连乘问题

问题： 给定 n 个矩阵 $\{A_1, A_2, \dots, A_n\}$ ，其中 A_i 为 $P_{i-1} \times P_i$ 阶矩阵， $i=1, 2, \dots, n$ 。

试确定计算矩阵连乘积的计算次序，使得矩阵链相乘需要的**总次数最少**。

输入： 向量 $P=\langle P_0, P_1, \dots, P_n \rangle$ ，其中 $P_0, P_1, \dots, P_n$ 为 n 个矩阵的行数与列数。

输出： 矩阵链乘法次数最少的加括号位置。



矩阵连乘问题

实例1: $P=\langle 10, 100, 5, 50 \rangle$

$A_1: 10 \times 100, A_2: 100 \times 5, A_3: 5 \times 50。$

乘法次序:

$(A_1 A_2) A_3: 10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500$

$A_1 (A_2 A_3): 100 \times 5 \times 50 + 10 \times 100 \times 50 = 75000$

第一种次序计算次数最少。

矩阵连乘问题

实例2: $P = \langle 25, 2, 40, 15, 30 \rangle$

$A: 25 \times 2, B: 2 \times 40, C: 40 \times 15, D: 15 \times 30$ 。

穷举法: 列举出**所有**可能的计算次序，并计算出每一种计算次序相应需要的数乘次数，从中找出一种数乘次数最少的计算次序。

Brute Force Algorithm



The brute-force method is to simply generate all possible routes and compare the distances. For a very small N , it works well, but it rapidly becomes absurdly inefficient when N increases.

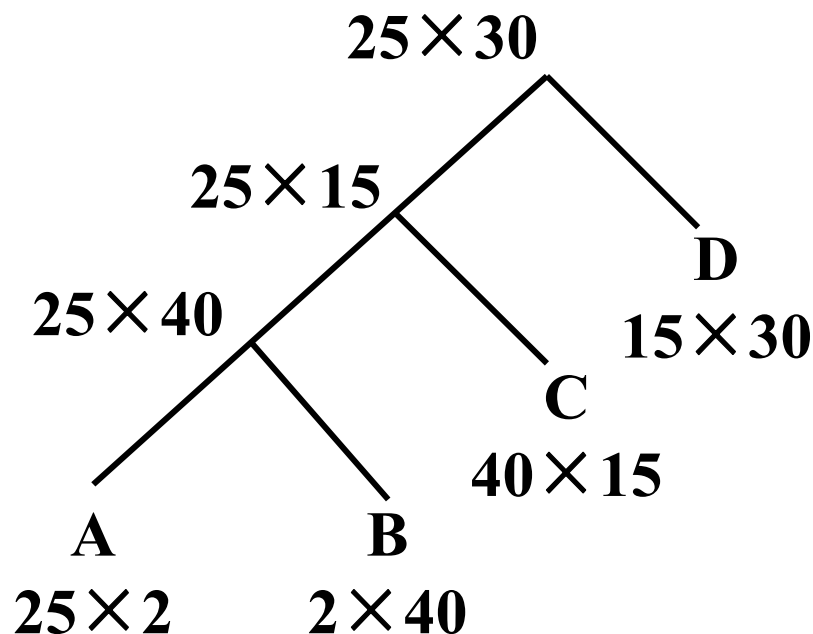
矩阵连乘问题

实例2: $P=\langle 25, 2, 40, 15, 30 \rangle$

A: 25×2 , **B:** 2×40 , **C:** 40×15 , **D:** 15×30 。

① $((AB)C)D$

$$\begin{aligned} &= 25 \times 2 \times 40 \\ &+ 25 \times 40 \times 15 \\ &+ 25 \times 15 \times 30 \\ &= \mathbf{28250} \end{aligned}$$



矩阵连乘问题

实例2: $P = \langle 25, 2, 40, 15, 30 \rangle$

A: 25×2 , **B:** 2×40 , **C:** 40×15 , **D:** 15×30 。

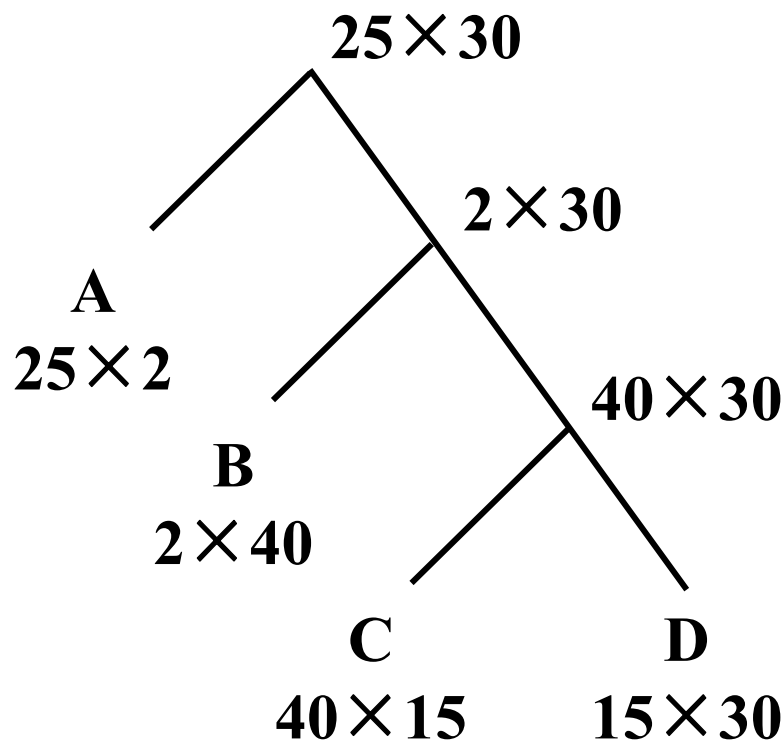
② $(A(B(CD)))$

$$= 40 \times 15 \times 30$$

$$+ 2 \times 40 \times 30$$

$$+ 25 \times 2 \times 30$$

$$= \mathbf{21900}$$



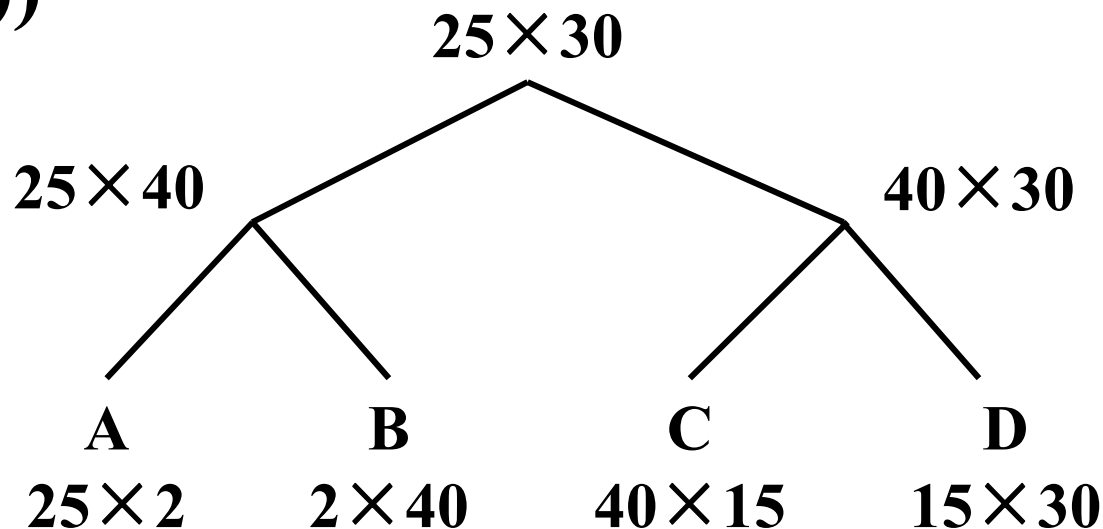
矩阵连乘问题

实例2: $P = \langle 25, 2, 40, 15, 30 \rangle$

$A: 25 \times 2, B: 2 \times 40, C: 40 \times 15, D: 15 \times 30$ 。

③ $((AB)(CD))$

= 50000

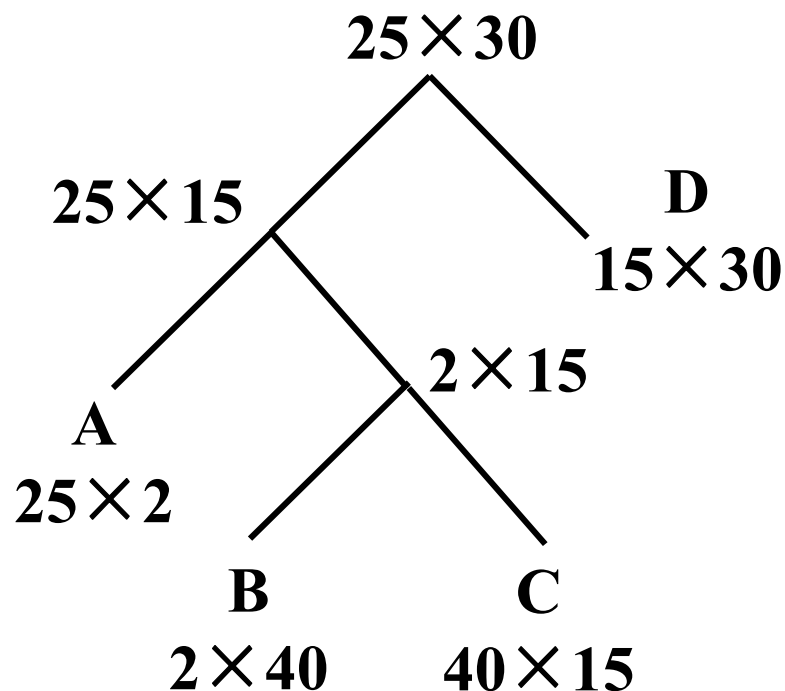


矩阵连乘问题

实例2: $P = \langle 25, 2, 40, 15, 30 \rangle$

A: 25×2 , **B:** 2×40 , **C:** 40×15 , **D:** 15×30 。

④ $((A(BC))D)$
= 13200

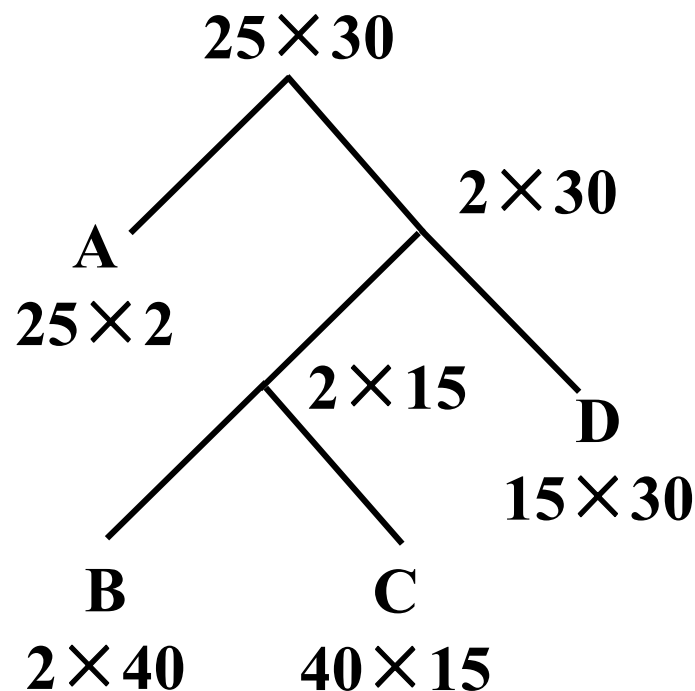


矩阵连乘问题

实例2: $P = \langle 25, 2, 40, 15, 30 \rangle$

$A: 25 \times 2, B: 2 \times 40, C: 40 \times 15, D: 15 \times 30$ 。

⑤ $(A((BC)D))$
 $= 3600$



矩阵连乘问题穷举法

复杂度分析

- 对 n 个矩阵连乘，设其不同的计算次序为 $P(n)$ 。
- 由于每种加括号方式都可以分解为两个子矩阵的加括号问题： $(A_1 \dots A_k)(A_{k+1} \dots A_n)$ 可以得到关于 $P(n)$ 的递推式如下：

$$P(n) = \begin{cases} 1 & n = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & n > 1 \end{cases}$$

Catalan数

即： $P(n) = C(n-1) = \frac{1}{(n-1)+1} \binom{2(n-1)}{n-1}$

矩阵连乘问题穷举法

- Stirling公式计算Catalan数
- 不失一般性，假设：

$$T(n) = \frac{1}{n+1} \times \binom{2n}{n} = \Omega\left(\frac{1}{n+1} \times \frac{(2n)!}{n! \times n!}\right)$$
$$= \Omega\left(\frac{1}{n+1} \times \frac{\sqrt{2\pi 2n} \left(\frac{2n}{e}\right)^{2n}}{\sqrt{2\pi n} \left(\frac{n}{e}\right)^n \times \sqrt{2\pi n} \left(\frac{n}{e}\right)^n}\right) = \Omega\left(\frac{2^{2n}}{n^{\frac{3}{2}}}\right)$$

复杂度是指数量级！！！！

矩阵连乘问题动态规划法

1. 分段：子问题划分

将矩阵连乘积 $A_i A_{i+1} \dots A_j$ 简记为 $A[i:j]$ ，这里 $i \leq j$

输入向量： $\langle P_{i-1}, P_i, \dots, P_j \rangle$

其最好划分的运算次数： $m[i, j]$

2. 分析：子问题的依赖关系

考察计算 $A[i:j]$ 的最优计算次序。设这个计算次序在矩阵 A_k 和 A_{k+1} 之间将矩阵链断开， $i \leq k < j$ ，即最优划分最后一次相乘发生在矩阵 k 的位置，则其相应完全加括号方式为 $(A_i A_{i+1} \dots A_k)(A_{k+1} A_{k+2} \dots A_j)$

矩阵连乘问题动态规划法

递推方程：

$m[i, j]$ ：得到 $A[i:j]$ 的最少的相乘次数。可递归定义为：

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \} & i < j \end{cases}$$

$$(A_i A_{i+1} \dots A_k) (A_{k+1} A_{k+2} \dots A_j)$$

$$P_{i-1} \times P_k$$

$$P_k \times P_j$$

该问题满足优化原则！

矩阵连乘问题动态规划法

3. 求解：计算最优值

- 对于 $1 \leq i \leq j \leq n$ 不同的有序对 (i, j) 对应于不同的子问题。因此，不同子问题的个数最多只有

$$\binom{n}{2} + n = \Theta(n^2)$$

- 由此可见，在递归计算时，许多子问题被重复计算多次。这也是该问题可用动态规划算法求解的又一显著特征。
- 用动态规划算法解此问题，可依据其递归式以自底向上的方式进行计算。在计算过程中，保存已解决的子问题答案。每个子问题只计算一次，而在后面需要时只要简单查一下，从而避免大量的重复计算，最终得到多项式时间的算法

递归实现动态规划的部分伪码

■ 算法1: $\text{RecurMatrixChain}(P, i, j)$

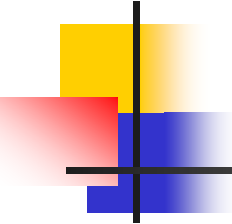
1. $m[i, j] \leftarrow \infty$
2. $s[i, j] \leftarrow i$
3. **for** $k \leftarrow i$ **to** $j-1$ **do**
4. $q \leftarrow \text{RecurMatrixChain}(P, i, k)$
 $+ \text{RecurMatrixChain}(P, k+1, j) + P_{i-1}P_kP_j$
5. **if** $q < m[i, j]$
6. **then** $m[i, j] \leftarrow q$
7. $s[i, j] \leftarrow k$
8. **return** $m[i, j]$

划分位置 k

子问题 $i:j$

找到了
更好的解

- 算法除了输出最优值数组 m 外，还输出记录最优断开位置的数组 s
- 这里没有写出算法的全部描述（递归边界）



算法分析

■ 时间复杂度的递推方程

$$T(n) \geq \begin{cases} O(1) & n=1 \\ \sum_{k=1}^{n-1} (T(k) + T(n-k) + O(1)) & n>1 \end{cases}$$

$$T(n) \geq O(n) + \sum_{k=1}^{n-1} T(k) + \sum_{k=1}^{n-1} T(n-k)$$

$$T(n) \geq O(n) + 2 \sum_{k=1}^{n-1} T(k)$$

可以证明还是一个指数函数

时间复杂度

- 数学归纳法证明: $T(n) \geq 2^{n-1}$
 - $n=2$, 显然为真
 - 假设对于任何小于 n 的 k , 命题为真

$$T(n) \geq O(n) + 2 \sum_{k=1}^{n-1} T(k)$$

代入归
纳假设

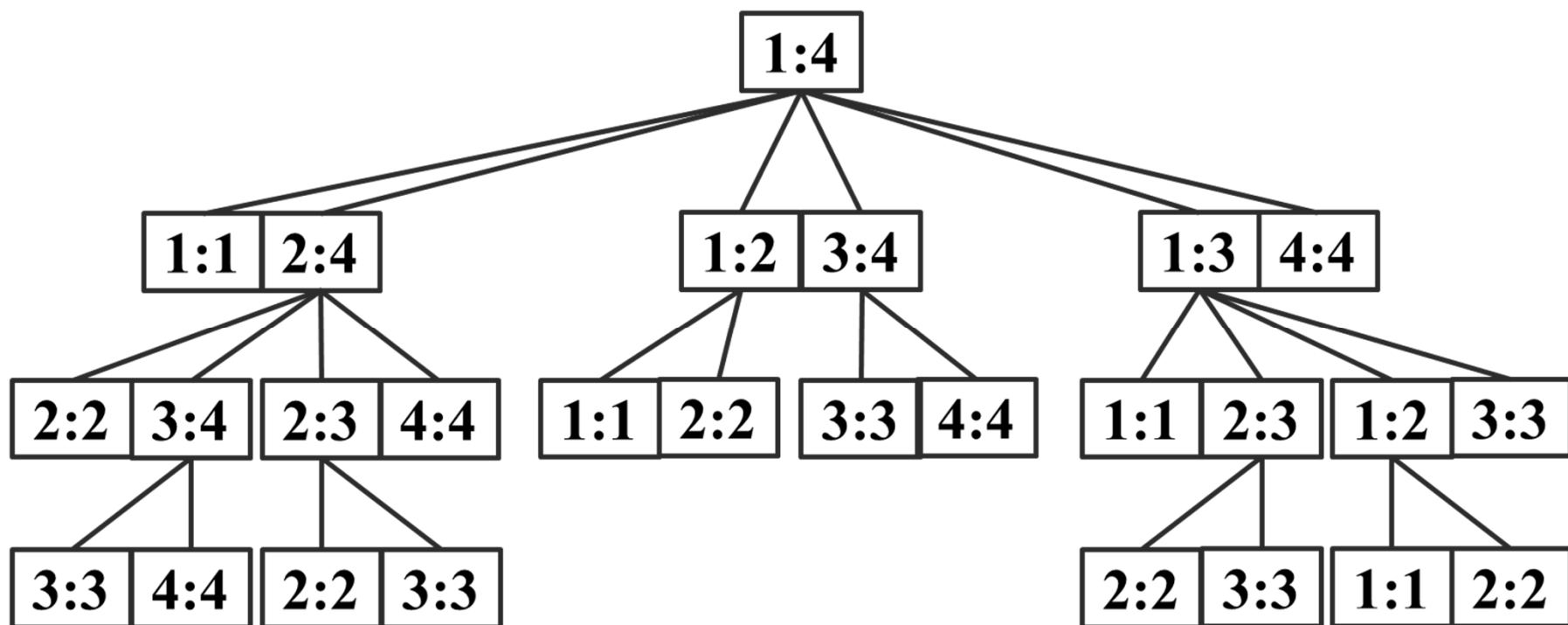
$$\geq O(n) + 2 \sum_{k=1}^{n-1} 2^{k-1}$$

等比数
列求和

$$= O(n) + 2(2^{n-1} - 1) \geq 2^{n-1}$$

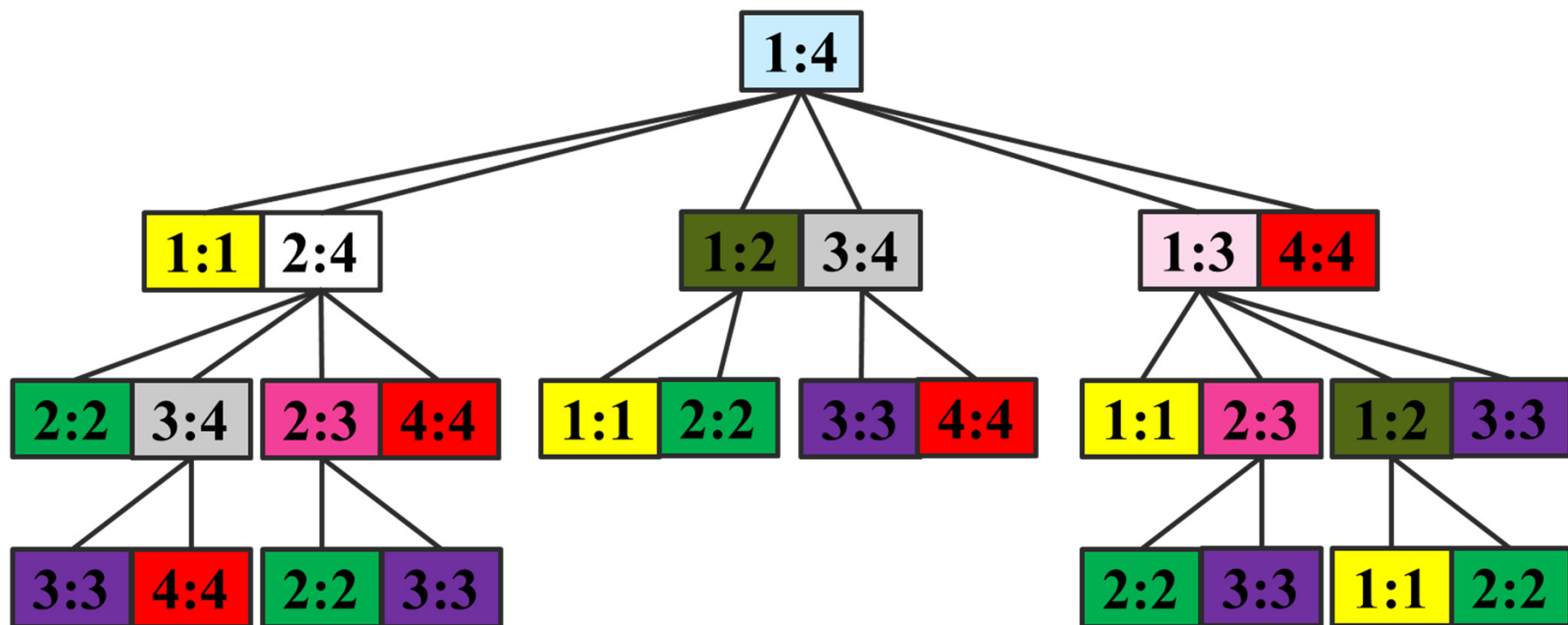
子问题的产生

用上述算法RecurMatrixChain(1,4)计算A[1:4]的递归树：



子问题的产生

用上述算法RecurMatrixChain(1,4)计算A[1:4]的递归树：



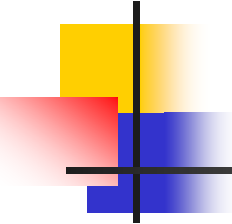
子问题的计数

边界	次数	边界	次数	边界	次数	边界	次数
1:1	4	1:2	2	1:3	1	1:4	1
2:2	5	2:3	2	2:4	1		
3:3	5	3:4	2				
4:4	4						

边界不同的子问题：**10**个

递归计算的子问题：**27**个

当 **$n=5$** 的时候，上面两个数值分别是**15**和**81**



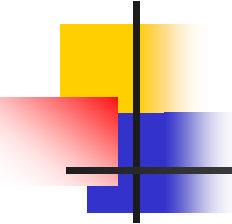
小结

- 与穷举法相比较，动态规划算法利用了子问题优化函数间的依赖关系，时间复杂度有所降低
- 动态规划算法的递归实现效率并不高，原因在于同一子问题多次重复出现，每次出现都需要重新计算一遍
- 还有没有改进的空间？

动态规划算法的迭代实现

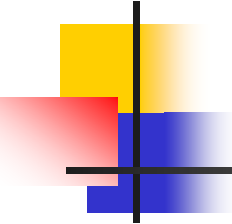
- **思想：**采用空间换时间策略，记录每个子问题首次计算结果，后面再用时就直接取值，每个子问题只计算一次！





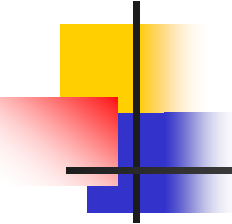
迭代计算的关键

- 每个子问题只计算一次
- 迭代过程
 - 从最小的子问题算起
 - 考虑计算顺序，以保证后面用到的值前面已经计算好了
 - 存储结构保存计算结果——备忘录
- 解的追踪
 - 设计标记函数标记每步的决策
 - 考虑根据标记函数追踪解的算法



矩阵链乘法不同子问题

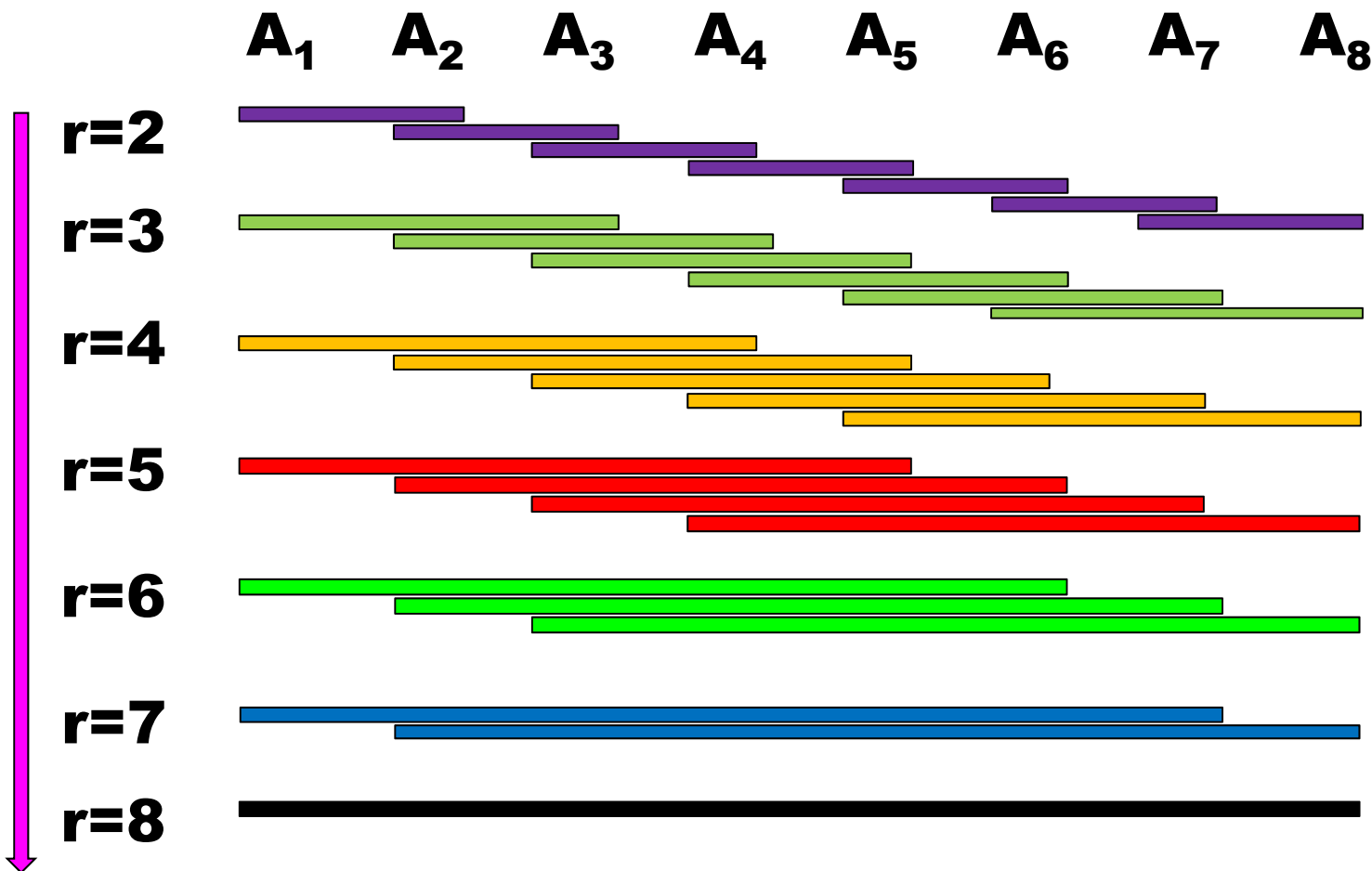
- **长度1**：只含1个矩阵，有 n 个子问题（不需要计算）
- **长度2**：含2个矩阵， $n-1$ 个子问题
- **长度3**：含3个矩阵， $n-2$ 个子问题
- ...
- **长度 $n-1$** ：含 $n-1$ 个矩阵，2个子问题
- **长度 n** ：原始问题，只有1个



矩阵链乘法迭代顺序

- 长度为1: 初值, $m[i,i]=0$
- 长度为2: $1:2, 2:3, 3:4, \dots, n-1:n$
- 长度为3: $1:3, 2:4, 3:5, \dots, n-2:n$
- ...
- 长度为 $n-1$: $1:n-1, 2:n$
- 长度为 n : $1:n$

$n=8$ 的子问题计算顺序



部分伪码

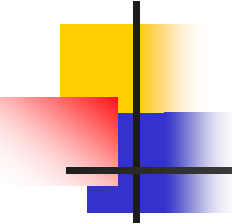
二维数组 m 与 s 为备忘录

■ 算法 MatrixChain(P, n)

1. 令所有的 $m[i,j]$ 初值为 0
2. for $r \leftarrow 2$ to n do // r 为链长
3. for $i \leftarrow 1$ to $n-r+1$ do // 左边界 i
4. $j \leftarrow i+r-1$ // 右边界 j
5. $m[i,j] \leftarrow m[i+1,j] + P_{i-1}P_iP_j$ // $k=i$
6. $s[i,j] \leftarrow i$ // 记录 k
7. for $k \leftarrow i$ to $j-1$ do // 遍历 k
8. $t \leftarrow m[i,k] + m[k+1,j] + P_{i-1}P_kP_j$
9. if $t < m[i,j]$
10. then $m[i,j] \leftarrow t$ // 更新解
11. $s[i,j] \leftarrow k$

遍历长度
为 r 子问题

遍历所有
划分



时间复杂度

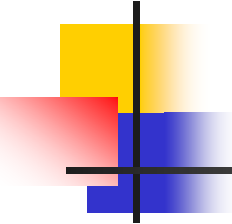
方法1. 根据伪码：行2, 3, 7都是 $O(n)$ ，循环执行 $O(n^3)$ 次，内部为 $O(1)$

$$T(n) = O(n^3)$$

方法2. 根据备忘录：估计每项工作量，然后求和。子问题有 $O(n^2)$ 个，确定每个子问题的最少乘法次数需要对不同划分位置比较，需要 $O(n)$ 时间。

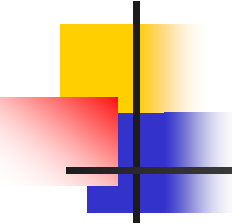
$$T(n) = O(n^3)$$

追踪解工作量 $O(n)$ ，总工作量 $O(n^3)$



实例

- **输入：** $P = \langle 30, 35, 15, 5, 10, 20 \rangle$
 $n = 5$
- **矩阵链：** $A_1 A_2 A_3 A_4 A_5$ ，其中
 $A_1 : 30 \times 35, A_2 : 35 \times 15, A_3 : 15 \times 5,$
 $A_4 : 5 \times 10, A_5 : 10 \times 20$
- **备忘录：** 存储所有子问题的最小乘法**次数**
及得到这个值的划分**位置**



备忘录 $m[i, j]$

■ $P = \langle 30, 35, 15, 5, 10, 20 \rangle$

$r=1$	$m[1,1]=0$	$m[2,2]=0$	$m[3,3]=0$	$m[4,4]=0$	$m[5,5]=0$
$r=2$	$m[1,2]=$	$m[2,3]=$	$m[3,4]=$	$m[4,5]=$	
$r=3$	$m[1,3]=$	$m[2,4]=$	$m[3,5]=$		
$r=4$	$m[1,4]=$	$m[2,5]=$			
$r=5$	$m[1,5]=$				